ONE MODEL. SIMULATE. FIT. OPTIMIZE.

Hands-On with Rumoca



James Goppert
Managing Director, PURT · Research Assistant Professor

Micah Condie
PhD student · School of Aeronautics and Astronautics

Purdue University · School of Aeronautics and Astronautics

MODEL
Declare the physics

FIT
Learn from data

OPTIMIZE
Plan the motion

Any modeling domain welcome · no aerospace background needed

CREATE CORE CODESPACE

Scan to create the core Codespace · materials linked below

Repository ↗ | **Workshop guide ↗** | **Create Codespace ↗**

Run the core. Explore your own question.

YOUR QUESTION. YOUR EXPERIMENT.

Make something you care about.

EXAMPLE AGENT PROMPT

“Use Three.js and Rumoca to make an interactive Artemis rocket launch simulation. Let me vary payload or throttle and check one result.”

Choose any problem you care about.

Demo: agent/tool + elapsed time + one check

THE PRIZE

FREE

CogniPilot stocking cap

Short demos
Audience hand vote

Enter by 12:40 · up to 5 demos · CHALLENGE.md · hosting optional

Challenge rules ↗ **Example prompts ↗** **Agent setup ↗**

Why Rumoca? One model, many questions.

A unified, declarative foundation for cyberphysical systems.

Lie group theory

Represent rotations and motion with their geometric structure.

Reachable sets

Bound possible behavior of cyberphysical systems under uncertainty.

Formal software verification

Connect model equations to explicit proof obligations.

System identification + digital twins

Fit physical parameters; compare simulation with experiments.

Cerebri RDD2: Modelica → eFMI C → Zephyr RTOS → real drone

Goal: mathematical guarantees for specified properties under explicit assumptions.

Rumoca compiler ↗ | Model library ↗ | Cerebri RDD2 ↗

How equations become executable programs

Modelica source → parse

AST
Parsed syntax + source locations

→

Flat
Expanded hierarchy + connections

→

DAE
Canonical equations; events + clocks

→

Solve
Scalar + tensor computation plans

resolve · typecheck · instantiate · flatten

structural preparation → solve lowering

↓ ↓

GaLEC
Fixed-sample control
eFMI / embedded C

JAX · CasADi
Torch · Lean
Continuous dynamics

Targets choose the IR they need.

Next: Solve → Jinja → Lean

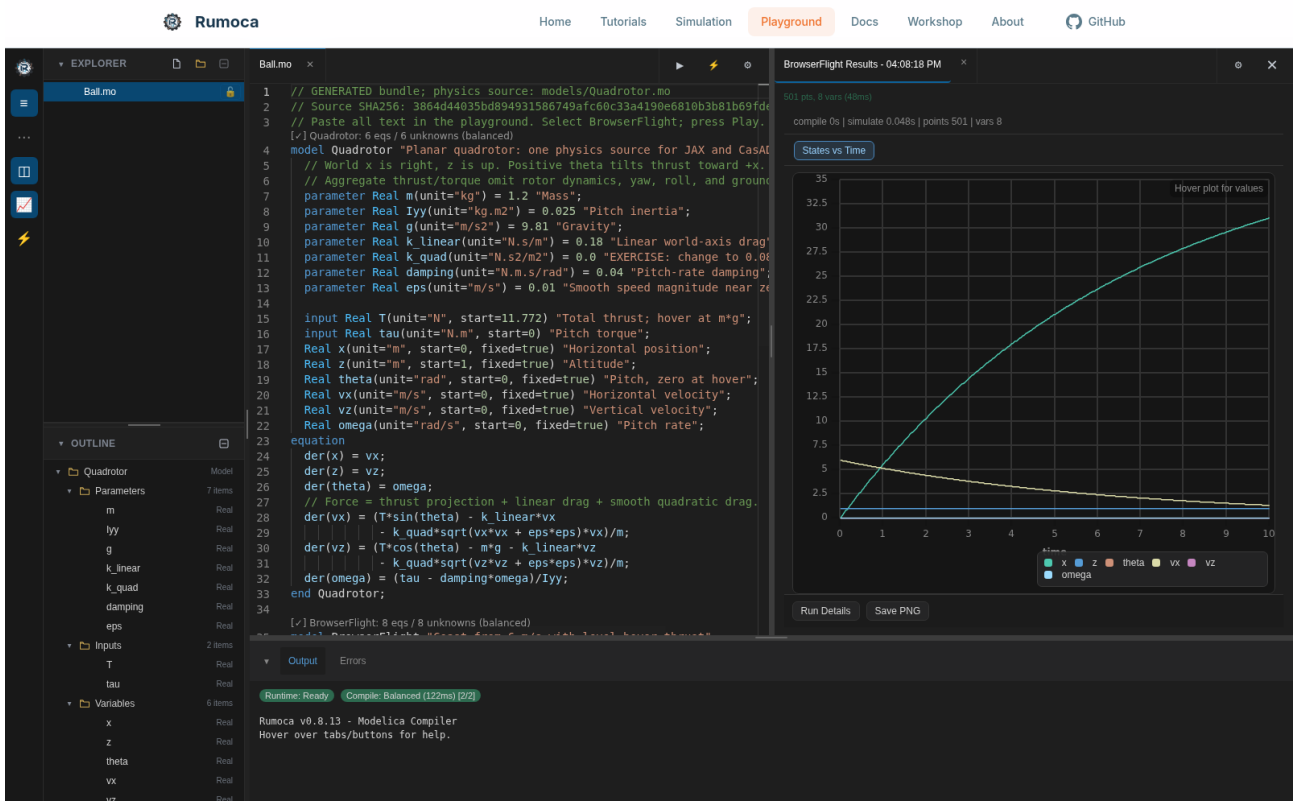
Same declared physics · inspectable stages · targets choose the IR they need.

Compiler pipeline ↗ | Custom targets ↗ | GaLEC / eFMI ↗

Everyone gets a browser plot

TRY THIS

1. Open the playground
2. Paste the complete browser bundle
3. Gear → BrowserFlight → Play



You should see BrowserFlight Results and a plot.

Open playground ↗

Browser model ↗

Choose a full-workshop route

01 Local container: Docker + Reopen in Container

02 Codespaces: same container; login and allowance

03 Local uv: optional native installation

04 Browser or pair: keep working during setup

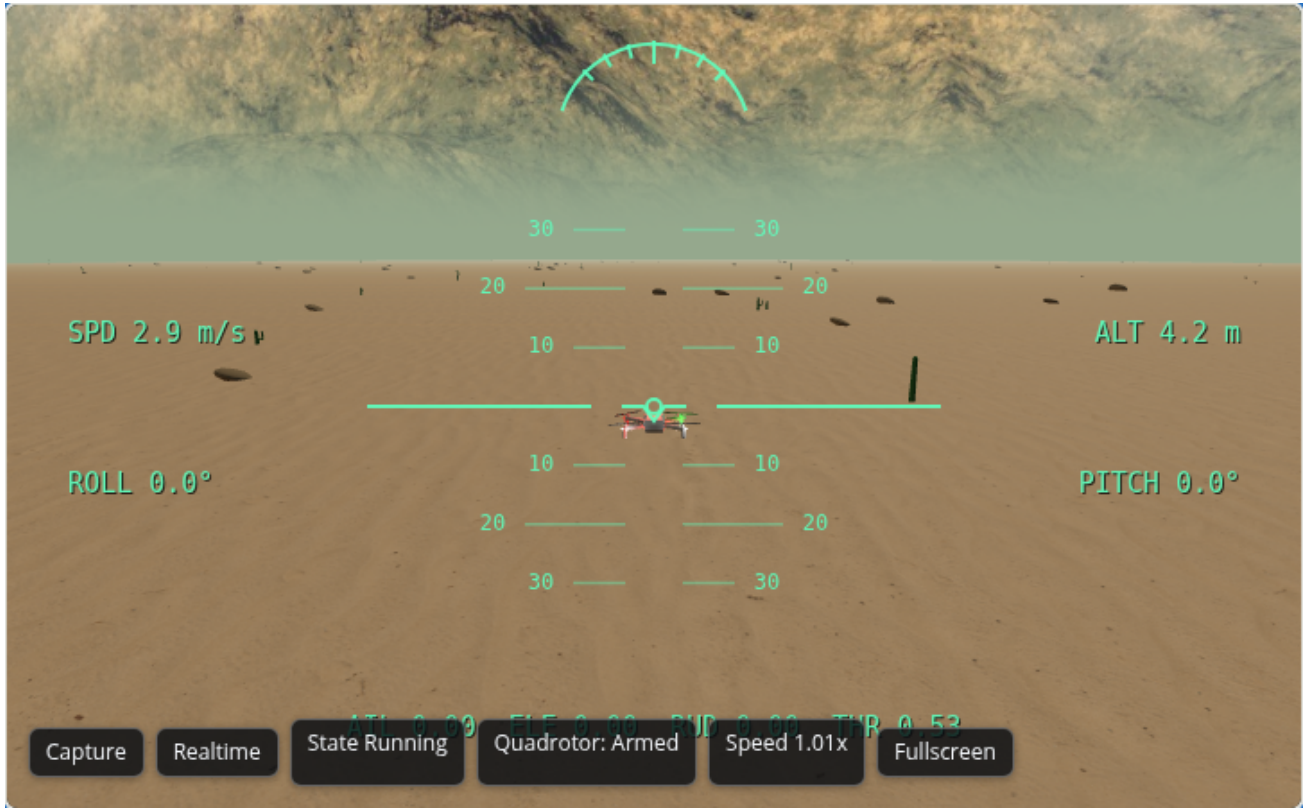
Switch route after ten minutes of setup trouble.

Local VS Code setup ↗ | Create Codespace ↗ | Recovery help ↗

Fly the user guide’s quadrotor

TRY THIS

1. Open Quadrotor SIL → Settings
2. Inspect the TOML → save → Simulate
3. Capture enables keys: Space arms; W / S changes throttle



VS Code: open .mo → lightning bolt creates TOML → Settings → Run.

Open Quadrotor SIL ↗ **Walkthrough ↗**

TOML: choose the model and the clock

rumoca-scenario.acro.toml · selected sections

```toml
[rumoca]
version = "1"
task = "simulate"

[model]
file = "QuadrotorSIL.mo"
name = "QuadrotorAcro"

[sim]
solver = "rk-like"
dt = 0.01
mode = "realtime"
```

1. file selects source; name selects the runnable class
2. dt = 0.01 s schedules 100 updates per simulated second
3. realtime paces the run; solver chooses the numerical method
4. Batch: set t_end; live: stop explicitly

Schema version "1" is the TOML format, not the Rumoca release.

Actual quadrotor TOML ↗ | Settings reference ↗

TOML: route controls into the model

Local state → input mapping → Modelica input

```
[locals.throttle]
type = "float"
default = 0.0

[input.gamepad.integrators.throttle]
source = "LeftStickY"
write = "throttle"
rate = 0.7
deadband = 0.1
clamp = [0.0, 1.0]

[signals.model_inputs]
stick_throttle = "local:throttle"
```

1. The stick changes persistent throttle; clamp keeps it within 0–1
2. W / S drives the keyboard integrator instead
3. Space toggles armed when throttle ≤ 0.05
4. signals.viewer names the channels the scene reads

Experiment: change rate 0.7 → 0.3; predict the throttle response.

Full input mappings ↗ **Copyable walkthrough ↗**

Three.js draws the simulated state

quadrotor_scene.js · excerpt from onFrame(api)

```
const get = api.get;
const px = get("px") ?? 0;
const py = get("py") ?? 0;
const pz = get("pz") ?? 0;

// Model: forward, left, up
// Three.js: map into x, y, z
const tx = -py, ty = pz, tz = px;

// Full script also transforms
// attitude and updates the mesh.
```

1. onInit: create/load meshes, lighting and persistent state
2. onFrame: read named signals and move the vehicle
3. Convert position and attitude together: $R_3 = A\,R\,A^{-1}$
4. Customize colors, trails or camera; keep physics in Modelica

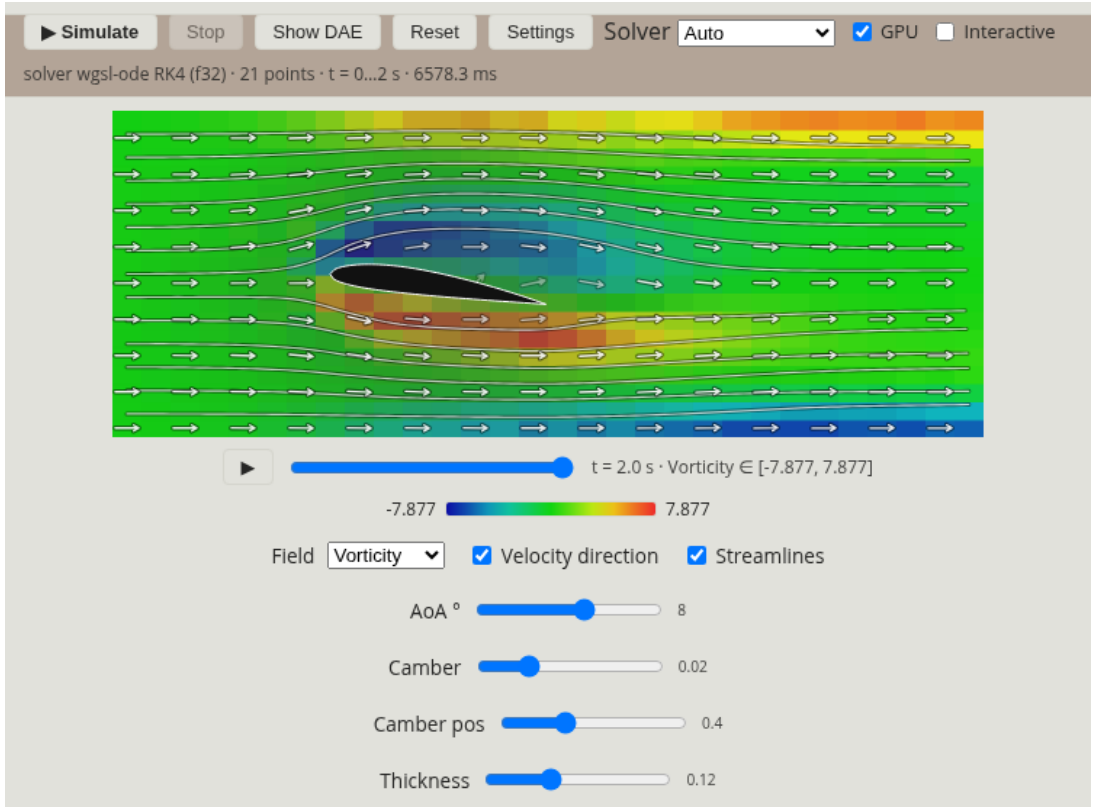
[transport.http].scene selects this JavaScript file.

Actual scene script ↗ **Scene + frame walkthrough ↗**

The user guide is a live modeling lab

TRY THIS

1. Edit StopTime: 30 → 2 s
2. Simulate; select the final frame
3. Switch vorticity / pressure



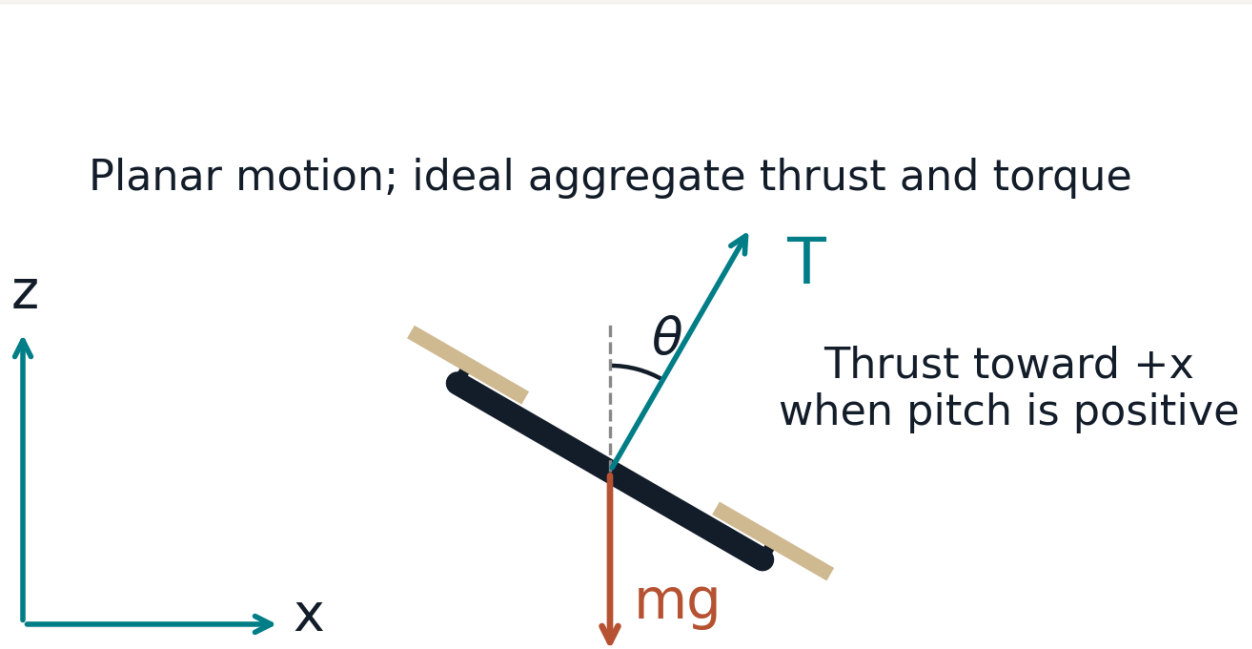
User-guide link and recovery: WORKSHOP.md · no install needed

Open NACA example ↗ **Explore user guide ↗**

A planar quadrotor we can inspect

READ THE MODEL

1. World x right; z up
2. Positive pitch tilts thrust toward +x
3. Hover: T = mg = 11.772 N



Find velocities, pitch rate, thrust and torque in the model.

Open vehicle model ↗

Workshop guide ↗

Your first engineering change: drag

Change k_quad = 0.0 to 0.08, save, then simulate.

$$D(v) = -k_{\text{linear}} v - k_{\text{quad}} \sqrt{v^2 + \varepsilon^2}\, v$$

Linear: N s/m Quadratic: N s²/m² ε = 0.01 m/s

Nonnegative coefficients ⇒ drag opposes motion

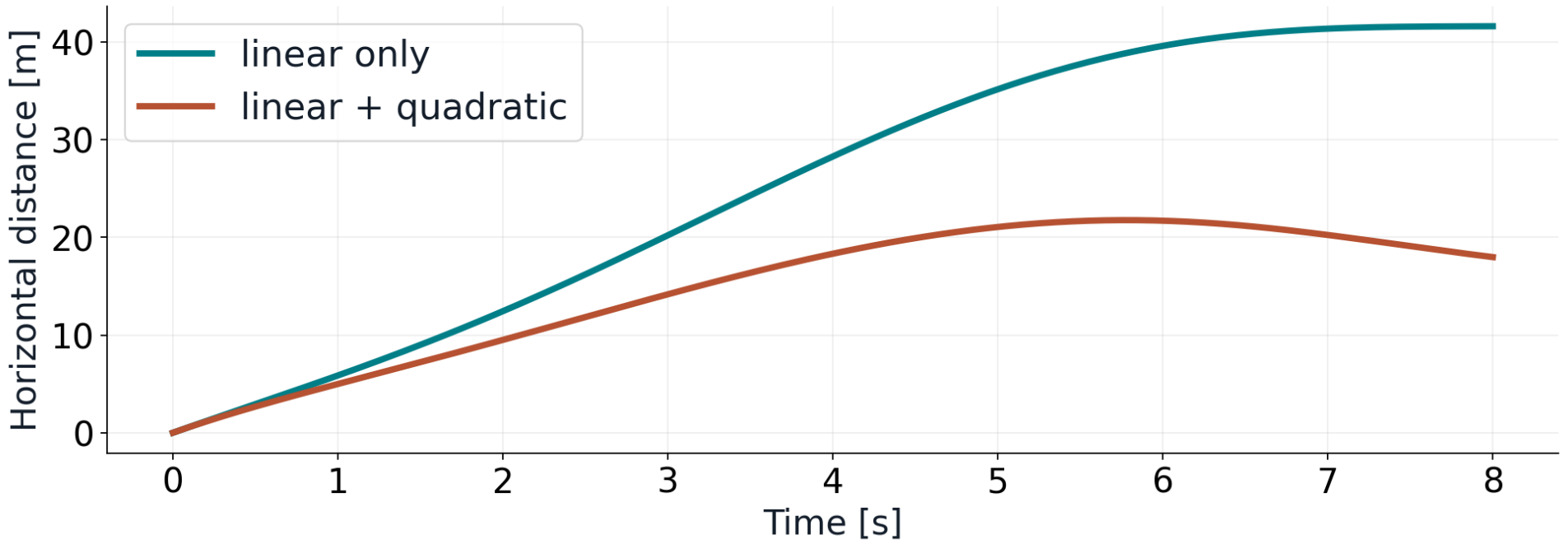
```
$ uv run --frozen python examples/simulate.py
```

What changes at high speed? What about negative velocity?

Workshop guide ↗ | Open playground ↗

More drag, less distance

Reference at 8 s: 41.60 m → 17.97 m



Compare your current default with the two reference cases.

Workshop guide ↗

Back at

11:10

10 MINUTES

Stretch. Ask a question. Keep your editor open.

KEEP YOUR PLACE

Keep your editor and terminal open.

Your simulation plot is the recovery point.

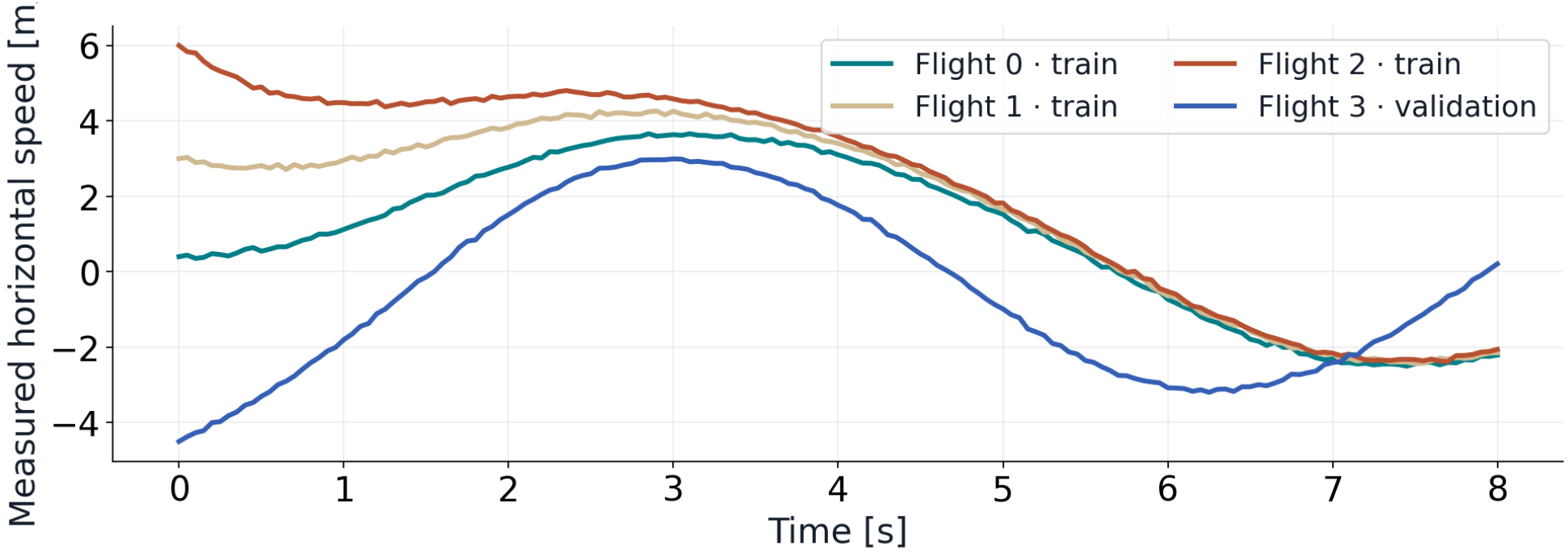
Next: learn drag from noisy data.

Worksheet and commands: WORKSHOP.md · scan the opening QR.

Workshop guide ↗

Known experiment. Unknown coefficients.

Three training flights; one held-out flight. 20 Hz, 8 s each.



Speed variation helps separate linear and quadratic drag.

Differentiate the prediction error



$$L = \mathrm{mean}\left[\left(\frac{\hat{y} - y}{\sigma}\right)^2\right]$$

$$k_l/k_{l,\mathrm{ref}} = e^a, \quad k_q/k_{q,\mathrm{ref}} = e^b$$

PHYSICAL COEFFICIENTS

k_l: linear drag [N s/m]
k_q: quadratic drag [N s²/m²]
k_{ref}: one corresponding SI unit

WHAT THE OPTIMIZER CHANGES

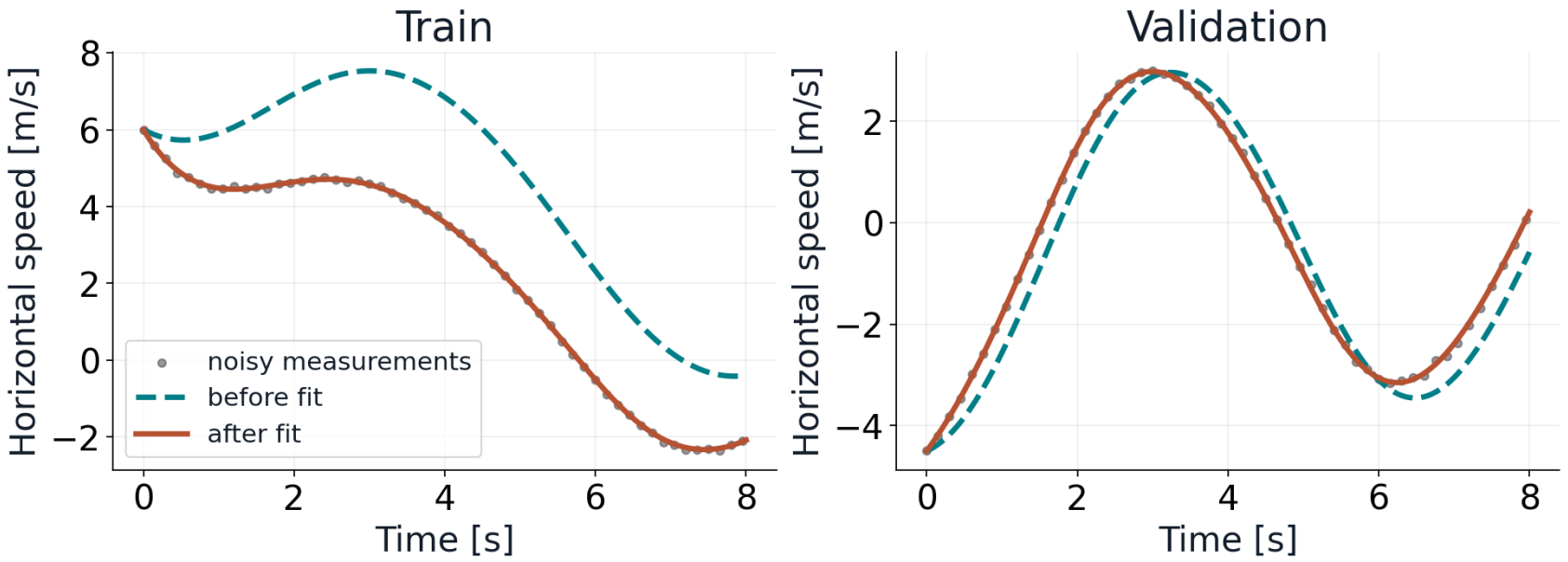
a, b: dimensionless log parameters
e ≈ 2.718; e^a and e^b keep drag positive

ŷ: prediction · y: measurement · σ: noise standard deviation · L: mean squared scaled error

JAX provides gradients. SciPy updates the coefficients.

Run the fit. Test the held-out flight.

Open fit_drag.png and fit_drag.json; compare before and after.




```
$ uv run --frozen python examples/fit_drag_jax.py
```

Two coefficients · lower loss · improved held-out prediction

Write a template. Generate checked equations.

1 TARGET.TOML · EXCERPT

```
ir = "solve"
[capabilities]
events = false
[[files]]
path = "Generated.lean"
template = "rhs.lean.jinja"
```

2 RHS.LEAN.JINJA · ACTUAL INPUT FIELDS

```
structure Control where
{% for name in solve.solve_layout.input_scalar_names %}
  {{ name }} : ℝ
{% endfor %}
```

Input names ["T", "tau"] → T : ℝ, tau : ℝ

`{% ... %}` repeats · `{{ ... }}` substitutes

3 INVOKE WITH THE SAME SESSION MODEL

```
model.codegen("templates/lean-proof").save_all("formal")
```

Check hover: level + still, T = m·g, zero torque → zero derivatives

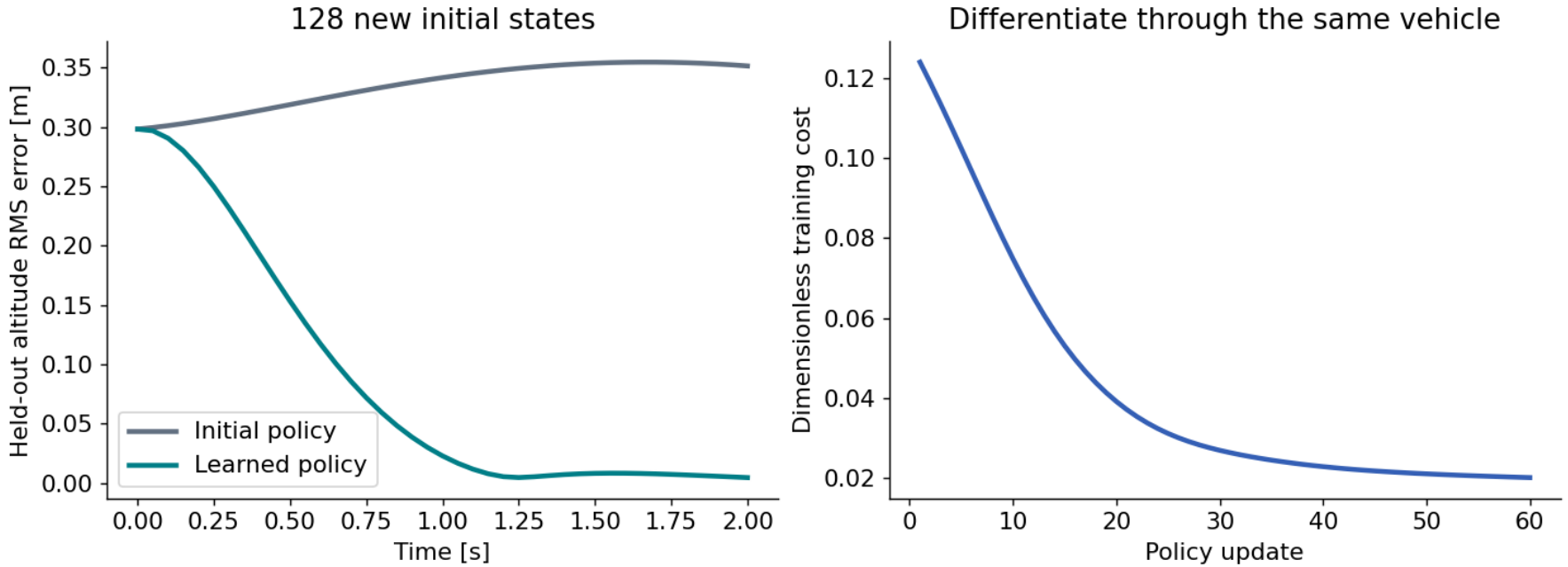
```
$ uv run --frozen python scripts/export_lean.py
```

17 exact-real theorems · compiler, template and numerical execution have separate checks.

Lean target.toml ↗ | Jinja template ↗ | Checked theorem ↗ | Try the walkthrough ↗

Make a target. Learn a policy.

target.toml → generated Torch → differentiable policy learning




```
$ uv run --frozen --group torch python scripts/learn_hover_torch.py
```

Held-out cost ≈ 0.116 → 0.021 · optional instructions: BACKENDS.md

Backend walkthrough ↗ | Torch target.toml ↗ | Lean Jinja target ↗

Back at

12:10

10 MINUTES

Stretch. Ask a question. Keep your editor open.

KEEP YOUR PLACE

Keep your fitted coefficients.

Next: plan a flight using your fit.

Your fit plot is the recovery point.

Same Modelica source; now compile to CasADi.

Move between hovers, respect limits

THE IDEA

1. (0, 1) m → (4, 2) m in 6 s
2. Track target; limit effort and control changes
3. Bound thrust, torque, pitch and altitude



T: 0–20 N τ: ±0.35 N m

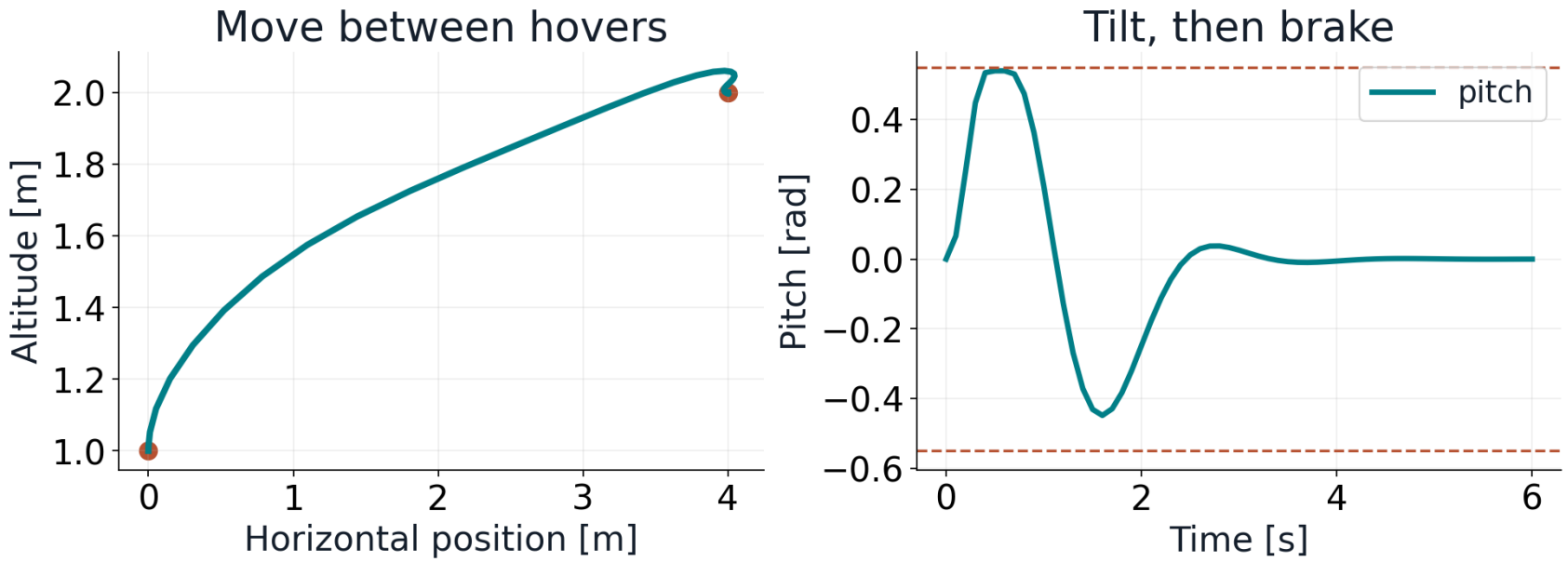
z: 0.25–4 m |θ| ≤ 0.55 rad

60 shooting intervals · node pitch margin · independent reintegration

Identified drag is an input to this optimization.

Solve. Inspect. Explain the braking.

Open optimization.png and optimization.json.

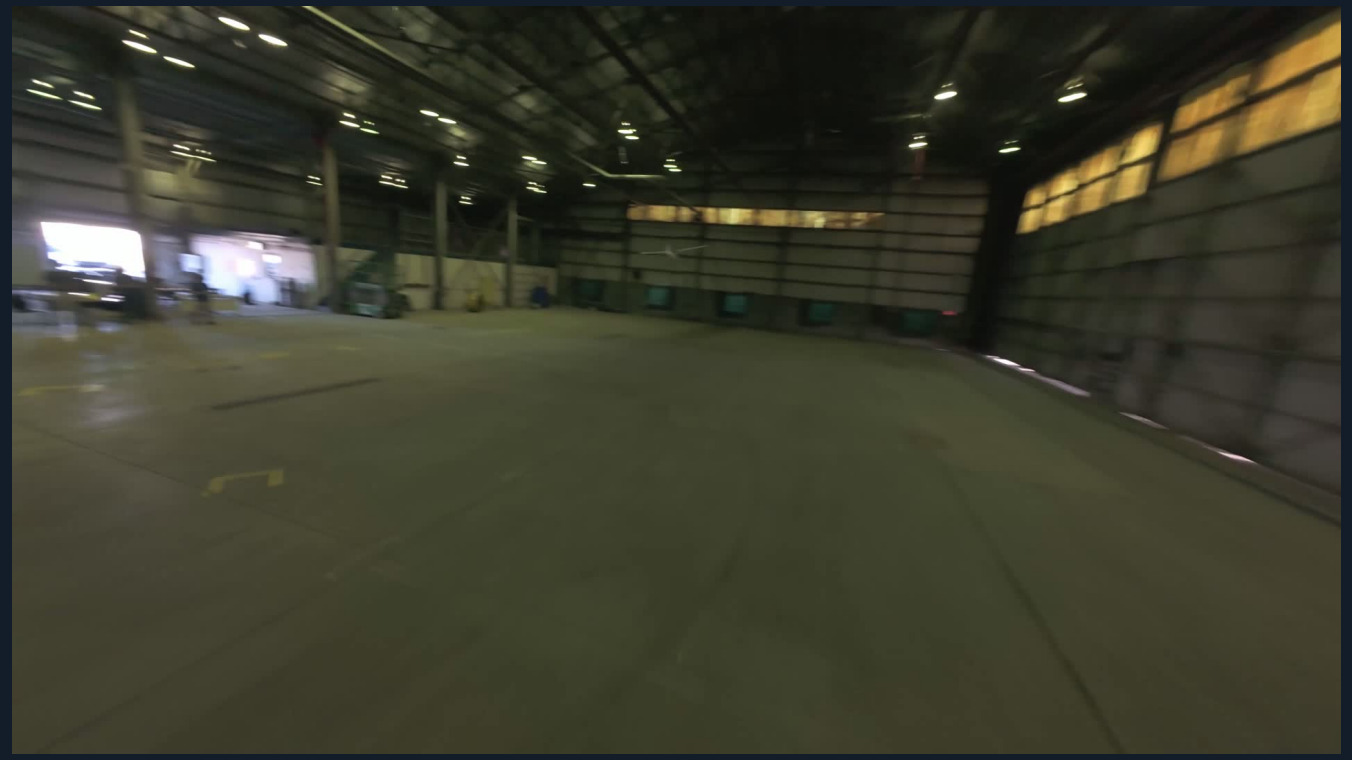



```
$ uv run --frozen python examples/optimize_casadi.py
```

Solve_Succeeded · limits respected · arrive near rest

Next up: pylon racing at PURT

The next competition will leverage Rumoca



PURT FPV chase · September 2025 · click video to play

FOLLOW THE NEXT COMPETITION

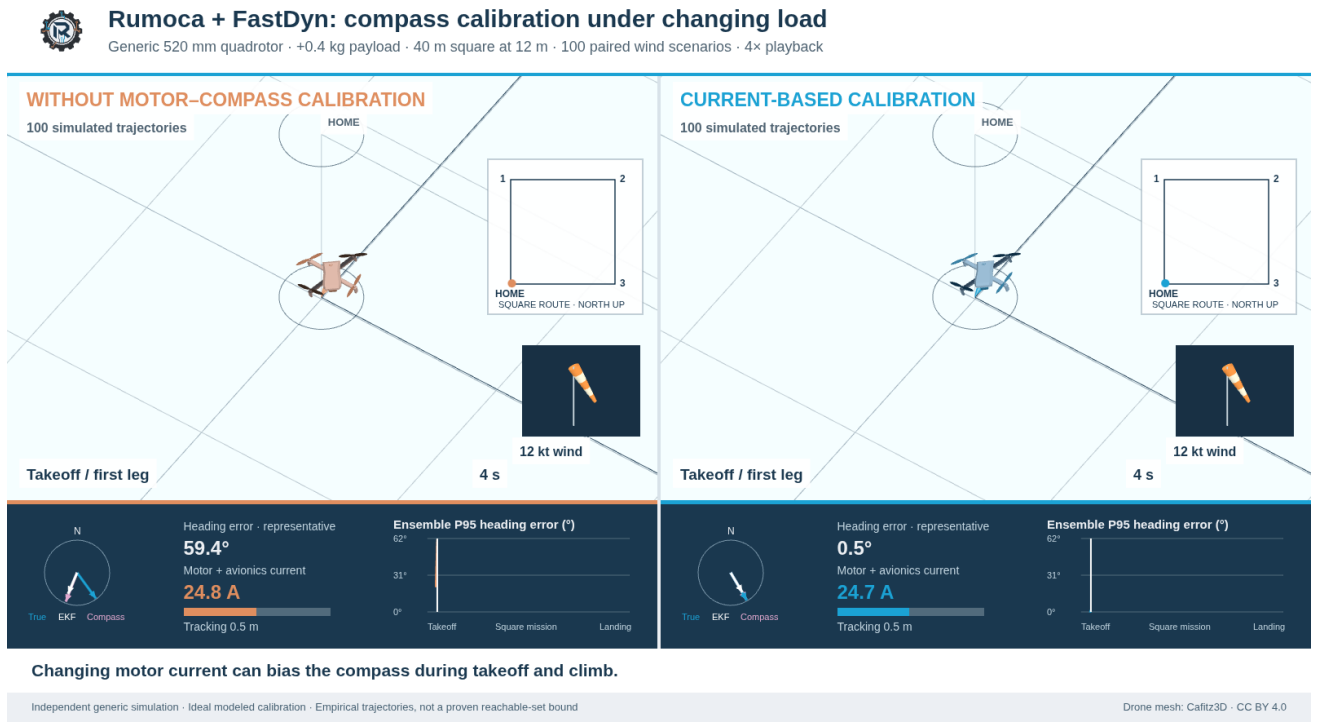
pylon-racing.geddes.rcac.purdue.edu

PURT pylon racing ↗ | **Watch race video ↗** | **Challenge rules ↗**

FastDyn: fly the firmware against declared physics

PHYSICS + ACTUAL FIRMWARE

1. Modelica → Rumoca FMI 3 plant
2. ArduPilot firmware → FastDyn / QEMU
3. Recorded: 100 paired wind scenarios
4. Try: 3 load trials in a FastDyn Codespace



Recorded research demo · separate from today's 3 trials

```
$ uv run --frozen python scripts/fastdyn_demo.py
```

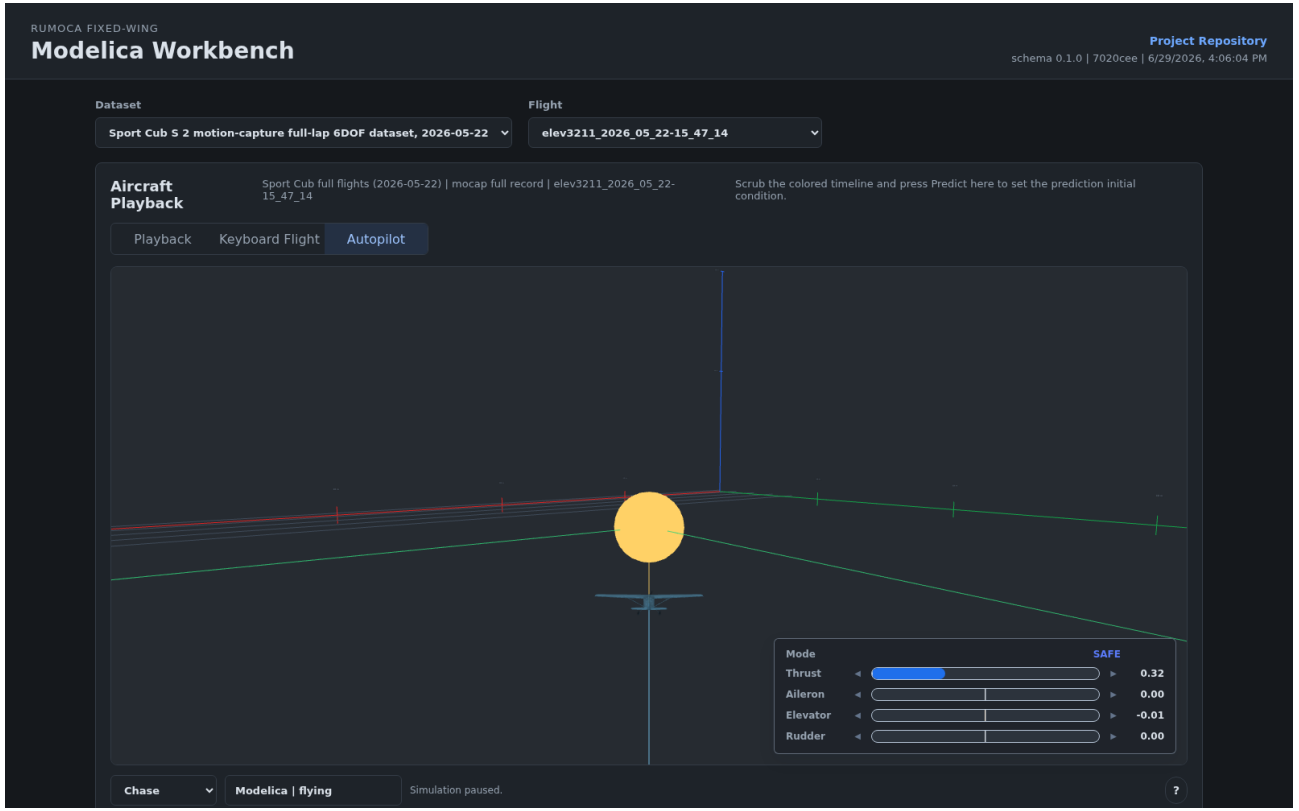
Choose the FastDyn container · open results/live/fastdyn/trajectories.png

FastDyn demo guide ↗ | BoardRunner paper ↗ | FastDyn source ↗ | Recorded Monte Carlo ↗

Explore the fixed-wing workbench

TRY THIS

1. Modelica flight simulation + autopilot
2. Try 50 Hz → 20 Hz, restart, then observe
3. Change a setting. Check what changes.



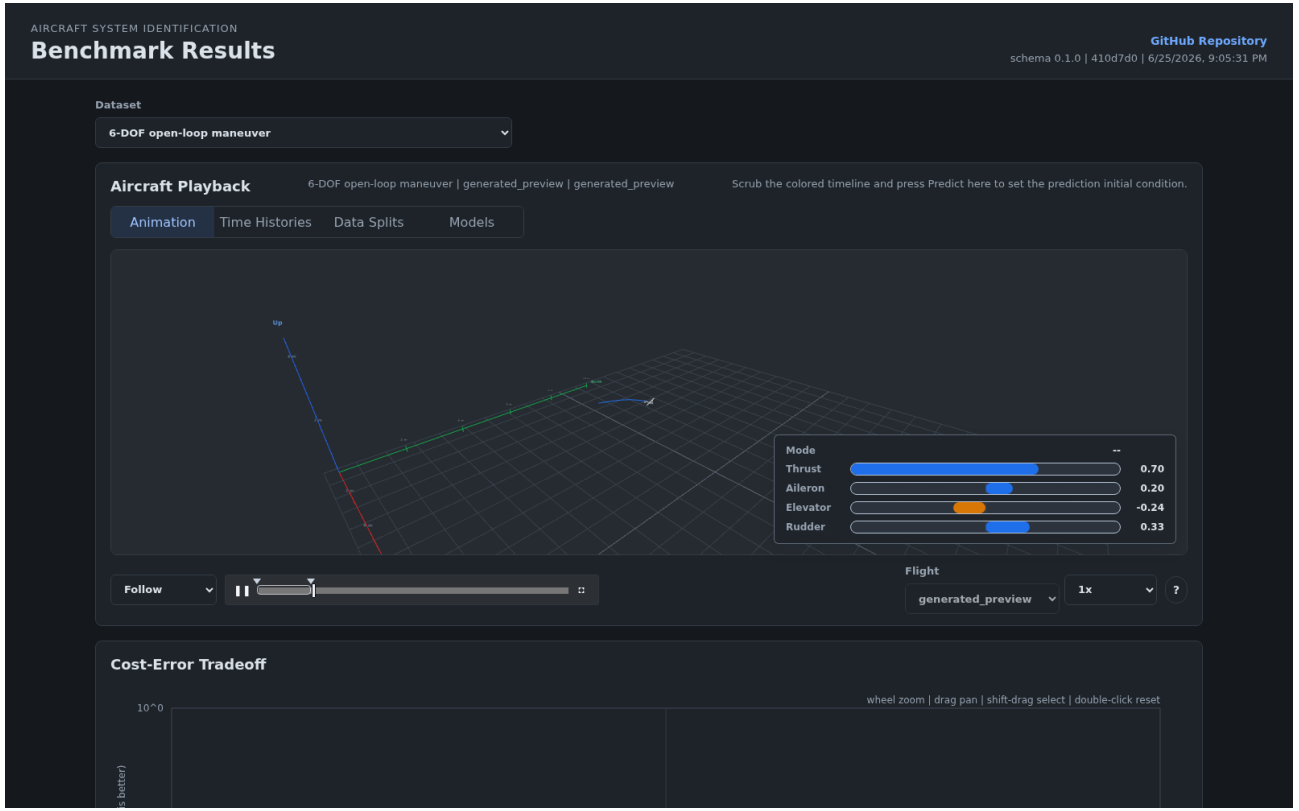
Live links + short experiments: BROWSER_APPS.md

Try fixed-wing ↗ | **Source code ↗** | **Experiment guide ↗**

Explore motion-capture identification

TRY THIS

1. Visualize vehicle data and model structure
2. Compare experiments and data splits
3. A viewer for exported benchmark results



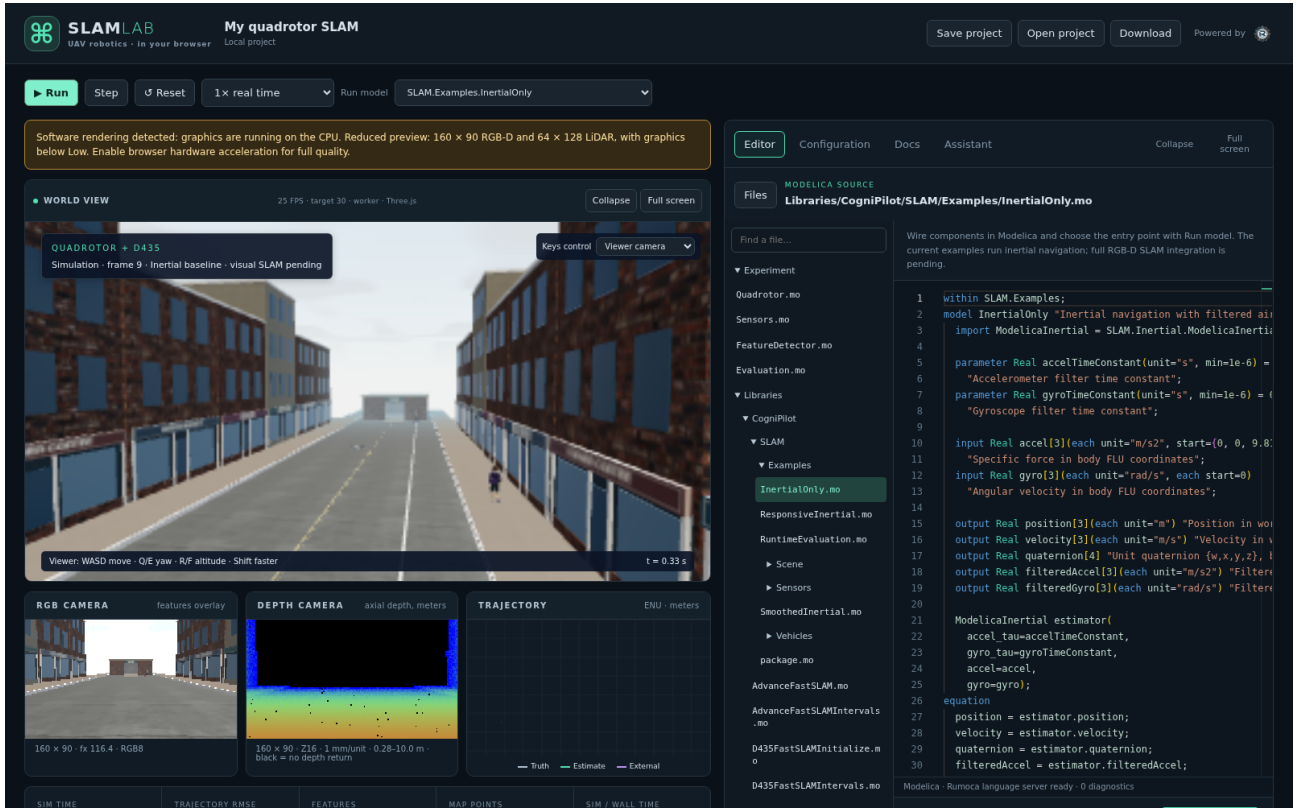
Explore exported identification results.

Try the app ↗ **Source code ↗** **Experiment guide ↗**

Explore SLAM Lab · in development

TRY THIS

1. Try the inertial estimation baseline
2. Inspect RGB, depth and trajectories
3. Visual SLAM and loop closure are in development



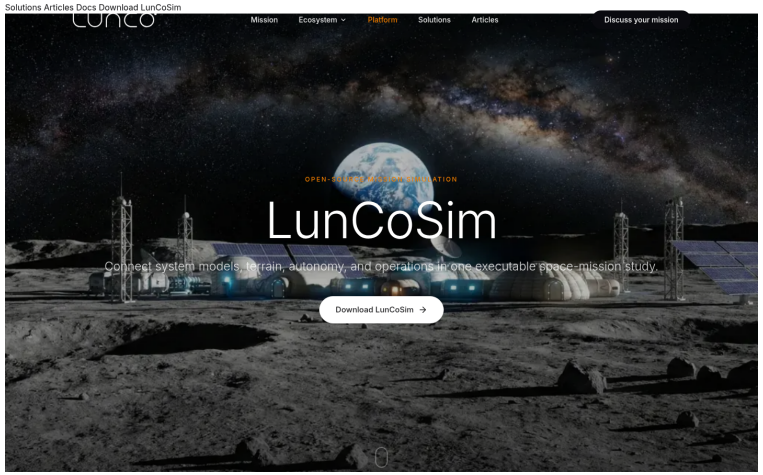
Start with the tested inertial baseline.

Try the app ↗ | Source code ↗ | Experiment guide ↗

A growing community builds with Rumoca

LunCoSim

Space-mission co-simulation

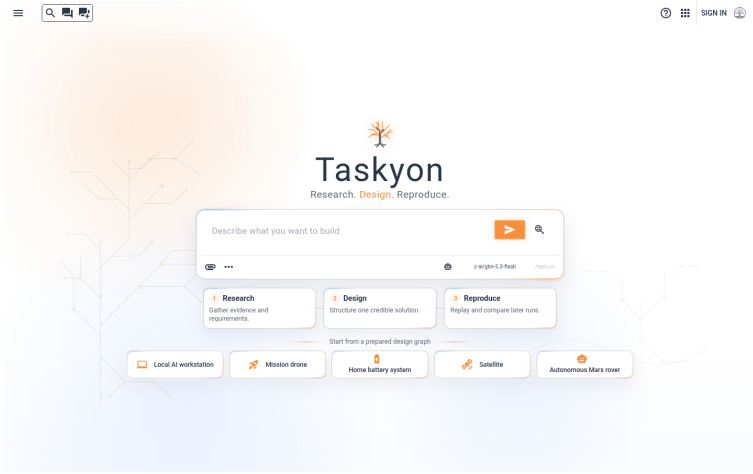


Rumoca-powered Modelica runtime

Public project-page preview

Taskyon

AI-assisted engineering workflows



Rumoca WASM Modelica tools

Public project-page preview

Where could declarative models fit in your tools?

LunCoSim simulator ↗ **LunCoSim source ↗** **Taskyon app ↗** **Taskyon source ↗**

Show your experiment. Vote for the cap.

5	45 s	1
MAX. PRESENTERS	EACH DEMO	VOTE PER PERSON

Show your change + one checked result

Name your tool. Share elapsed work time.

Audience favorite wins a free CogniPilot stocking cap

Random draw if more than 5 ready entries → audience hand vote

Challenge rules ↗ Optional gallery ↗

Bring your model.

Join the Rumoca research community.

1. Bring a model or a question from your own domain
2. Contribute a validation case, model library or compiler improvement
3. Interested in research collaboration? Talk with James or Micah

jgoppert@purdue.edu

James Goppert · Micah Condie

Email James with your problem, model and one result to check together.

Email James ↗ | Rumoca compiler ↗ | Model library ↗